

Laboratorio di Programmazione

Lezione 8: Stringhe (2)

1	NOTA: leggere testo da standard input	2
2	Ristampa testo	3
3	Qual è l'output?	4
4	Statistiche testo	5
5	Testo invertito	6
6	Numero nascosto	7
7	Linguaggio farfallino	8
8	Garibaldi	9
9	Spaziatura	10
10	Il cifrario di Cesare	11

1 NOTA: leggere testo da standard input

Negli esercizi seguenti dovreste leggere da standard input un testo, anche su più righe, la cui fine è segnalata dal simbolo EOF (End-Of-File). Potete usare il seguente codice, il cui funzionamento preciso verrà spiegato più avanti:

```
1 scanner := bufio.NewScanner(os.Stdin)
2 for scanner.Scan() {
3     testo += scanner.Text() + "\n"
4 }
```

Ovviamente vanno importati `bufio` e `os`. Sostanzialmente, ogni iterazione del ciclo legge una riga dello standard input, fermandosi quando appare EOF, segnalato dal fatto che `scanner.Scan()` ritorna `true`. Siccome `scanner.Text()` elimina il `\n` dalla fine della riga, `\n` va ri-aggiunto (a meno che non vogliate collassare le righe in una riga sola).

Il simbolo EOF può essere inserito da tastiera con la combinazione di tasti CTRL+D; viene invece inserito automaticamente quando lo standard input viene letto da file, tramite redirectione. Vedete l'esercizio [2](#).

2 Ristampa testo

Scrivere un programma che:

- legga da standard input un testo su più righe.
- termini la lettura quando incontra EOF (End-Of-File; da tastiera, Ctrl+D).
- dopo aver terminato la lettura, ristampi il testo letto.

Il programma deve definire e utilizzare una funzione `LeggiTesto()` string che legge e ritorna il testo di input, come descritto sopra.

Esempio d'esecuzione:

```
$ go run ristampa.go
Inserisci testo (termina con CTRL+D):
Testo di prova
disposto su più righe
```

```
Alcune possono essere anche righe vuote
Testo letto:
Testo di prova
disposto su più righe
```

Alcune possono essere anche righe vuote

NOTA. Invece di ri-scrivere ripetutamente da tastiera, potete creare un file e usare la redirectione dello standard input o il pipelining (nota: ^D significa che è stato premuto CTRL+D):

```
$ cat >> hello.txt
Hello World!
Also---Bye!
^D
```

```
$ go run ristampa.go < hello.txt
Hello World!
Also---Bye!
```

```
$ cat hello.txt | go run ristampa.go
Hello World!
Also---Bye!
```

3 Qual è l'output?

```
1 package main
2
3 import (
4     "fmt"
5 )
6
7 const NumeroLettere = 'Z' - 'A' + 1
8
9 func main() {
10
11     var carattere rune = 'Z'
12
13     carattere += 2
14     carattere -= 'A'
15     carattere %= NumeroLettere
16     carattere += 'A'
17
18     fmt.Printf("%c\n", carattere)
19 }
```

4 Statistiche testo

Scrivere un programma che legge da standard input un testo, anche su più righe, finché non incontra EOF (End-Of-File; da tastiera, CTRL+D), e poi stampi:

1. il numero di parole
2. la lunghezza media delle parole

Nota: una parola è una stringa formata da sole lettere; numeri, punteggiatura, spaziature, e quant'altro non fanno quindi parte di parole. Leggete la documentazione della funzione `unicode.IsLetter()`.

Il programma deve definire e utilizzare le seguenti funzioni:

- `LeggiTesto()` string come nell'esercizio 2
- `StatisticheParole(T string) (int, int)` calcola il numero di parole nel testo T e la loro lunghezza totale.

Esempio d'esecuzione:

```
$ go run statistiche.go
Inserisci testo (termina con Ctrl+D):
Testo di prova

su cui calcolare le statistiche.
Test 01: prova.prova0prova
Statistiche:
Numero parole: 12
Lunghezza media: 4.833333333333333
```

5 Testo invertito

Scrivere un programma che legga da standard input un testo, anche su più righe, finché non incontra una riga vuota. A quel punto il programma deve stampare il testo, ma invertito (cioè partendo dall'ultimo simbolo e terminando col primo).

Il programma deve definire e utilizzare le seguenti funzioni:

- `LeggiTesto() string`, come nell'esercizio 2, ma opportunamente adattato.
- `InvertiStringa(s string) string` che restituisce l'inversa di `s`.

Esempio d'esecuzione:

```
$ go run stringainvertita.go
```

Inserisci un testo su più righe (termina con riga vuota):

Testo di prova

disposto su due righe

Testo invertito:

ehgir eud us otsopsid

avorp id otseT

6 Numero nascosto

Scrivere un programma che legga da standard input una riga di testo, T , e calcoli il suo *numero nascosto* N , che si ottiene concatenando le cifre presenti in T , nell'ordine in cui appaiono (se T non contiene alcuna cifra allora $N = 0$). Il programma deve poi stampare il numero $2N$. Usate le funzioni `strconv.Atoi()` e `unicode.IsDigit()`.

Il programma deve definire e utilizzare le seguenti funzioni:

- `LeggiTesto()` string che legge una riga da standard input;
- `NumeroNascosto(T string) (int, error)` che restituisce il numero nascosto di T , e l'eventuale codice di errore restituito da `strconv.Atoi()`.

Esempio d'esecuzione:

```
$ go run numero_nascosto.go
Ci8ao 97com3 va?
Doppio del numero nascosto: 17946 (8973 * 2)
```

```
$ go run numero_nascosto.go
Ch3 831 t3mp0
Doppio del numero nascosto: 766260 (383130 * 2)
```

```
$ go run numero_nascosto.go
c140n3
Doppio del numero nascosto: 2806 (1403 * 2)
```

```
$ go run numero_nascosto.go
nessun numero nascosto
```

7 Linguaggio farfallino

Nel linguaggio farfallino, ciascuna vocale non accentata *v* viene sostituita da una sequenza di tre caratteri *vf**v*. Per esempio, *a* viene sostituita da *afa*, mentre *e* da *efe*, e così via; la stessa cosa per le maiuscole. Ogni altro carattere resta invece inalterato. Scrivere un programma che legge da standard input un testo, anche su più righe e terminando quando incontra EOF, e lo ristampi in linguaggio farfallino. Il programma deve definire e utilizzare le seguenti funzioni:

- `LeggiTesto()` `string`, come nell'esercizio 2.
- `Farfallino(c rune)` `string` che converte il carattere *c* in farfallino.
- `ConvertiInFarfallino(T string)` `string` che converte il testo *T* in farfallino.

Suggerimento: per verificare se un carattere è una vocale potete usare `if`, `switch`, o la funzione `strings.ContainsRune()`.

Esempio d'esecuzione:

```
$ go run farfallino.go
Inserisci un testo su più righe (termina con CTRL+D):
Questo è un testo di prova
da trasformare IN ALFABETO FARFALLINO
Risultato:
Qufuefestofe èfè ufun tefestofe difi profovafa
dafa trafasfoformafarefe IFIN AFALFAFABEFETOFO FAFARFALLIFINOFO
```


8 Garibaldi

Scrivere un programma che:

- legga da standard input un testo su più righe, terminando quando viene inserita una riga vuota;
- ristampi il testo letto sostituendo ogni vocale con la lettera *u*.

Il programma deve definire e utilizzare le seguenti funzioni:

- `LeggiTesto() string` come nell'esercizio 5
- `TrasformaCarattere(c rune) rune` che ritorna la lettera *u* se *c* è una vocale, e ritorna *c* stesso altrimenti
- `Garibaldi(s string) string` che trasforma *s* usando `LeggiTesto()`

Esempio d'esecuzione:

```
$ go run garibaldi.go
```

Inserisci un testo su più righe (termina con riga vuota):

```
Garibaldi fu ferito  
fu ferito in una gamba,  
Garibaldi che comanda  
che comanda i bersaglier
```

Risultato trasformazione:

```
Gurubuldu fu furutu  
fu furutu un unu gumbu,  
Gurubuldu chu cumundu  
chu cumundu u bursugluur
```

9 Spaziatura

Scrivere un programma che:

- legga da standard input un testo su più righe, terminando quando incontra EOF;
- ristampi il testo letto, ma inserendo uno spazi tra ogni coppia di caratteri che sono entrambi *non* di spaziatura (suggerimento: usate la funzione `unicode.IsSpace()`).

Il programma deve definire e utilizzare le seguenti funzioni:

- `LeggiTesto() string` come nell'esercizio 2.
- `Spazia(s string) string` che spazia `s` come descritto sopra.

Suggerimento:

Esempio d'esecuzione:

```
$ go run spaziatura.go
Inserisci un testo su più righe (termina con CTRL+D):
Testo di prova
da stampare con spaziatura
Risultato:
T e s t o d i p r o v a
d a s t a m p a r e c o n s p a z i a t u r a
```

10 Il cifrario di Cesare

Giulio Cesare usava, per le sue corrispondenze riservate, un codice cifrato nel quale ogni lettera viene sostituita dalla lettera che la segue di tre posti nell'alfabeto:

originale	A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
cifrato	D E F G H I J K L M N O P Q R S T U V W X Y Z A B C

Più in generale, un cifrario di Cesare è un codice nel quale ogni lettera del testo viene spostata di un numero fisso k di posti; l'intero k è detto *chiave di cifratura*.

Scrivere un programma che:

- legga da standard input un intero k e poi un testo, terminando quando incontra EOF.
- stampi il testo, cifrato secondo il cifrario di Cesare con chiave k .

Il programma deve definire e utilizzare le seguenti funzioni:

- `LeggiNumero()` `int` che legge da standard input un intero;
- `LeggiTesto()` `string` come nell'esercizio 2
- `CifraCarattere(c rune, k int)` `rune` che cifra il carattere `c` con chiave `k`. Notate che il cifrario deve preservare le maiuscole/minuscole, e inoltre se `c` non è una lettera allora deve restituire `c` stesso.
- `CifraTesto(T string, k int)` `string` che cifra il testo `T` usando la chiave `k`

Esempio d'esecuzione:

```
$ go run cifrario_cesare.go
Inserisci un numero: 1
Inserisci un testo su più righe (termina con CTRL D):
Testo di esempio
diviso su righe diverse
Testo cifrato:
Uftup ej ftfnpj
ejwjtp tv qsjhif ejwfstf
```

```
$ go run cifrario_cesare.go
Inserisci un numero: -2
Inserisci un testo su più righe (termina con CTRL D):
AbC

dEf
Testo cifrato:
YzA

bCd
```